САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 2

Взаимодействие с внешним API

Выполнил:

Губанов Егор

Группа K3440

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Доработать сайт «Вместе» из первой лабораторной работы по варианту 3 «Платформа для фитнес-тренировок и здоровья». Подключить страницы к API с помощью fetch, создать моковое API средствами JSON Server и добавить авторизацию пользователя.

Ход работы

Я сохранил семь страниц сайта: каталог, вход, регистрацию, личный кабинет, описание тренировки, комнату занятия и блог. Для оформления использовал HTML, CSS и Bootstrap, а для работы с данными – JavaScript.

Сервер запускается на Node.js командой npm start. Данные хранятся в db.json: users содержит пользователей и их планы, trainings – тренировки, articles – статьи. JSON Server предоставляет адреса для получения и изменения этих данных.

В api.js я вынес общий запрос fetch. Запросы GET получают занятия, статьи и профиль. POST отправляет данные регистрации и входа. PATCH сохраняет личный план. Ответ сервера преобразуется из JSON в объект JavaScript.

При входе JSON Server Auth проверяет почту и пароль и возвращает токен JWT. Браузер сохраняет токен в localStorage и передаёт его в заголовке Authorization. Сервер разрешает пользователю читать и менять только собственную запись. Хеш пароля в ответ профиля не передаётся. Фильтры каталога передаются в параметрах запроса. На странице тренировки можно выбрать день недели и добавить занятие в план. Личный кабинет показывает сохранённые занятия и прогресс. Статьи блога также загружаются с сервера. В тренировках добавлены видеозаписи, а в комнате сохранён таймер этапов.

Проверка работы

В Safari проверены регистрация, вход, сообщение о неверном пароле, фильтрация, добавление занятий в план и сохранение результата. После перехода обратно в личный кабинет прогресс остался сохранён. Отдельными запросами проверено, что без токена доступ закрыт, а чужой профиль прочитать или изменить нельзя.

Скриншоты сайта

Основные страницы и действия показаны в браузере Safari.

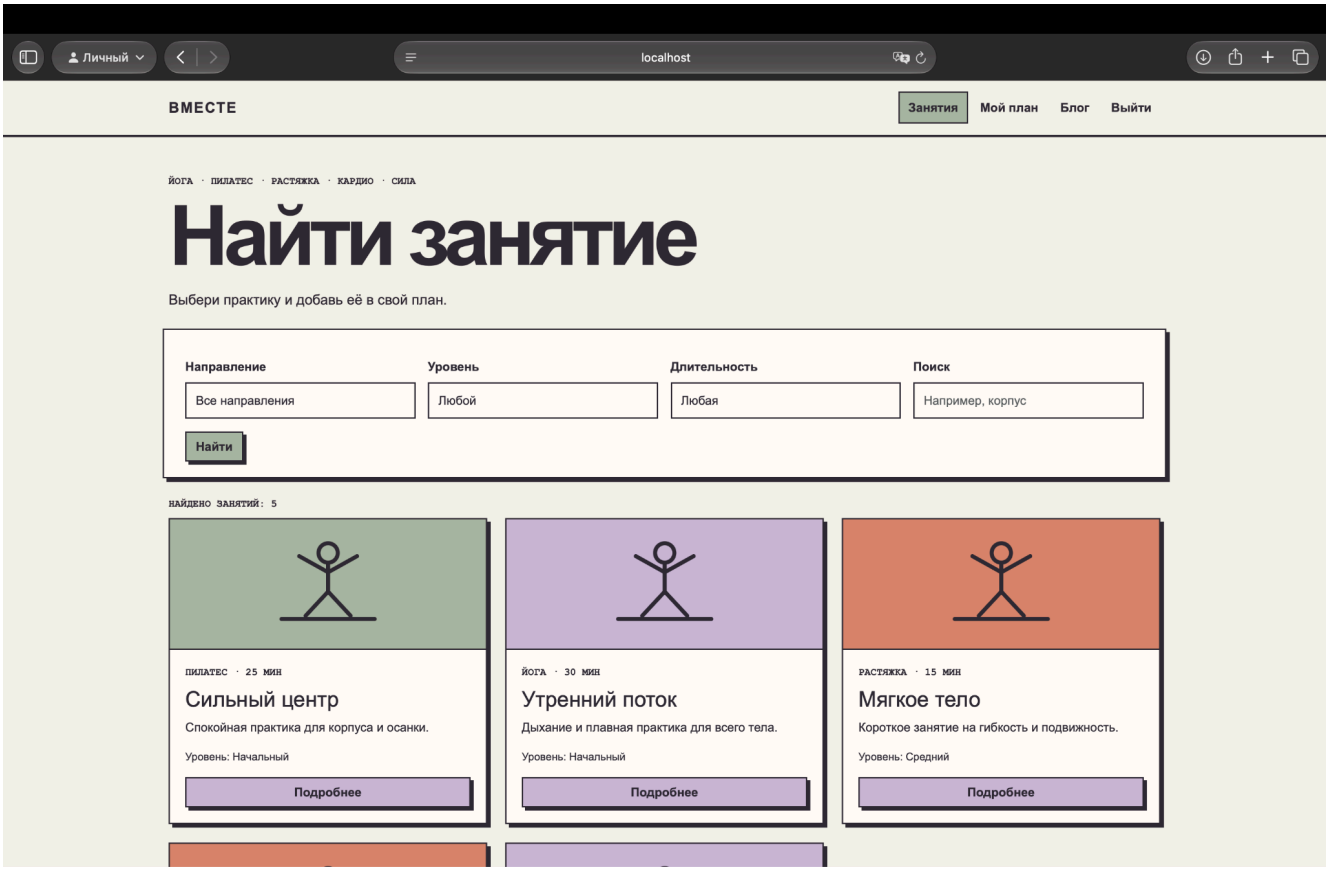


Рисунок 1. Каталог занятий с данными из API

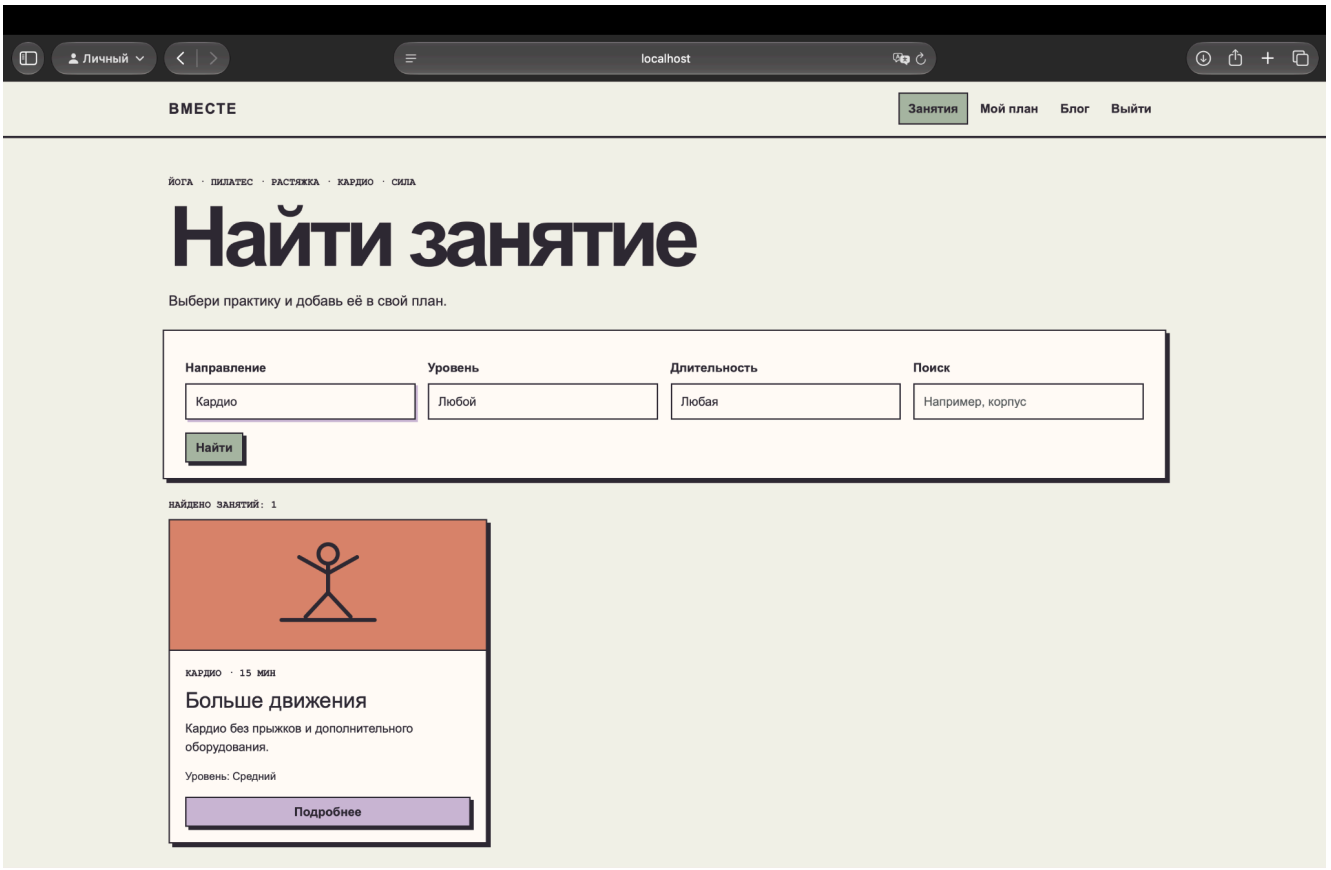


Рисунок 2. Фильтрация каталога по направлению Кардио

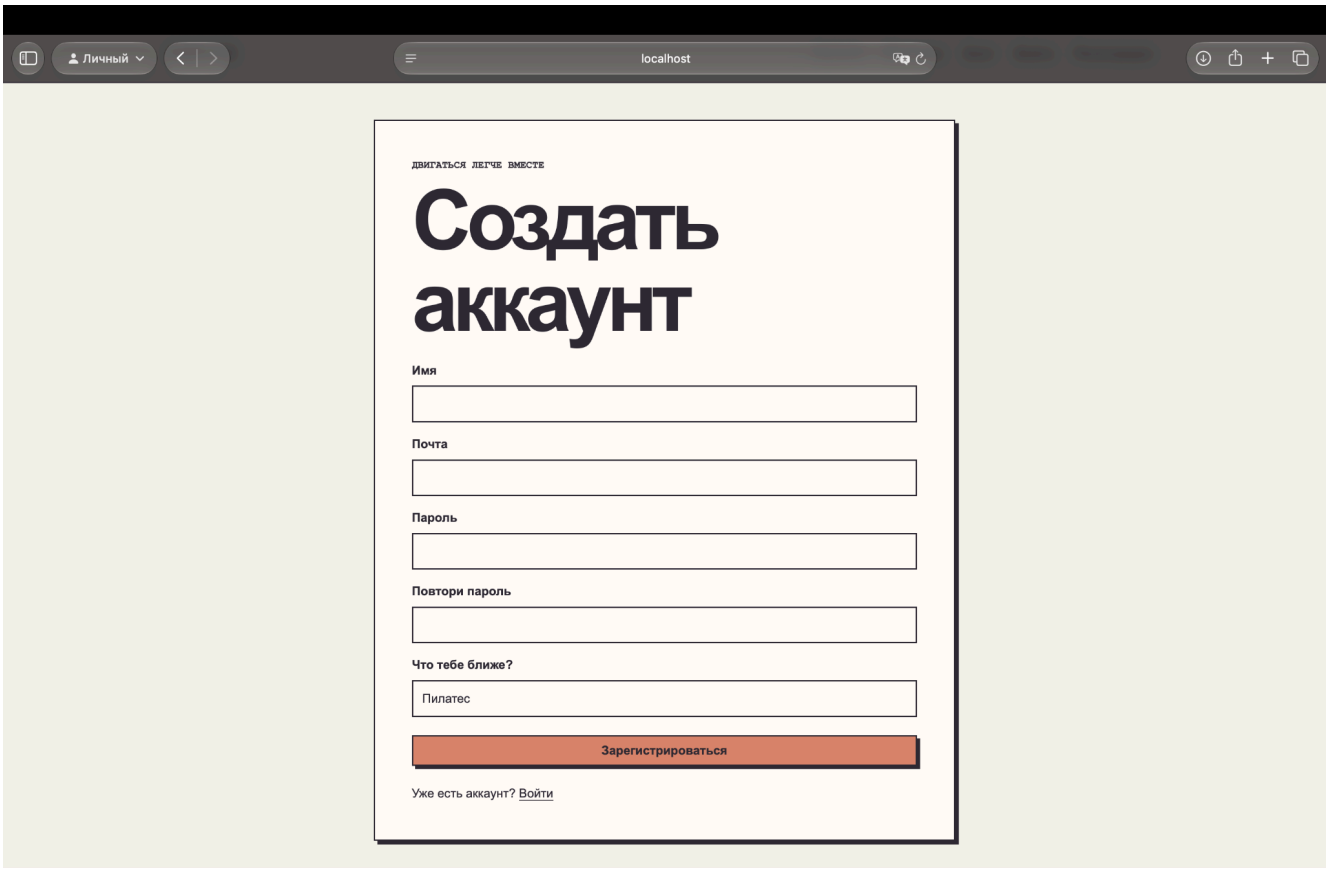


Рисунок 3. Форма регистрации пользователя

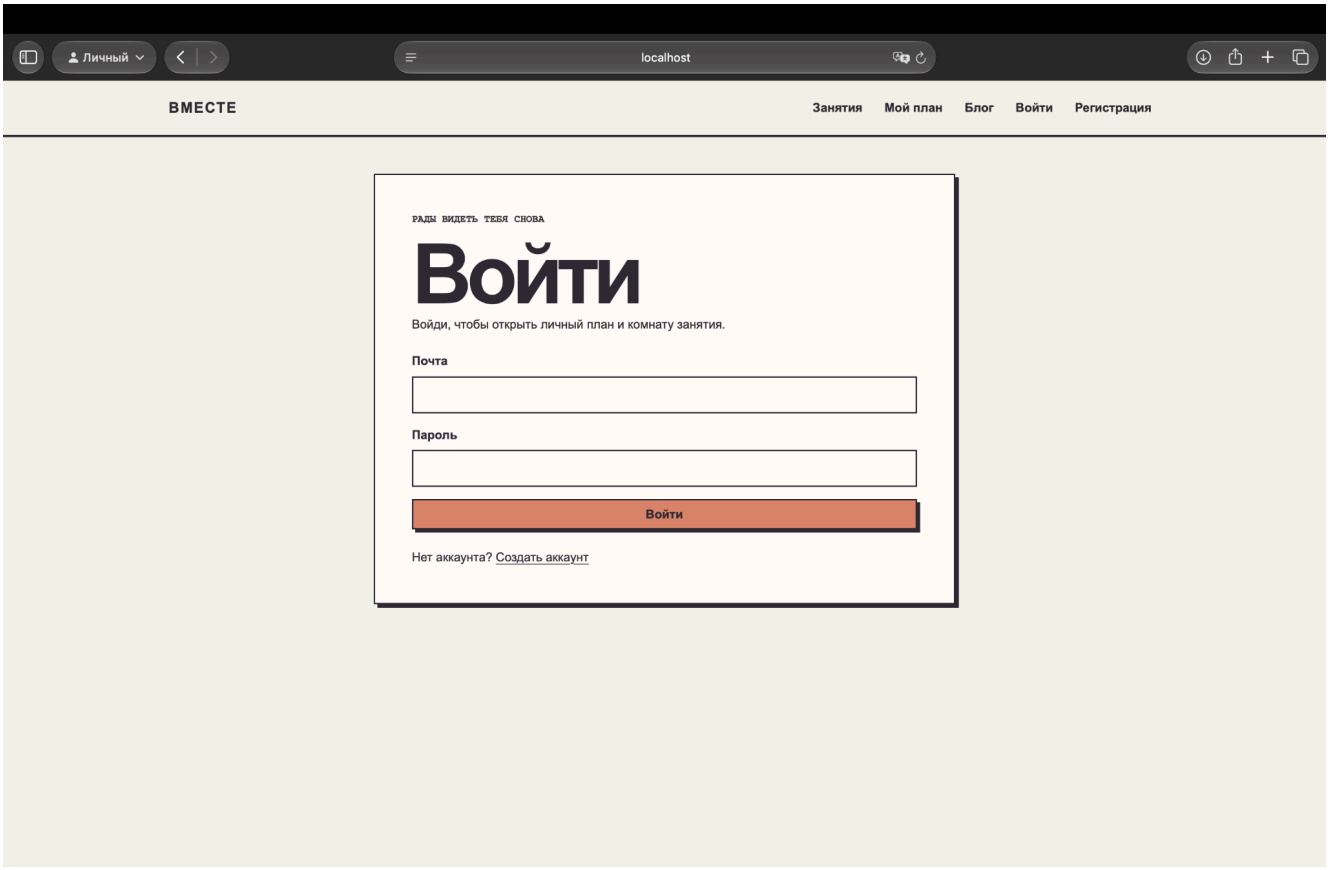


Рисунок 4. Сообщение при вводе неверного пароля

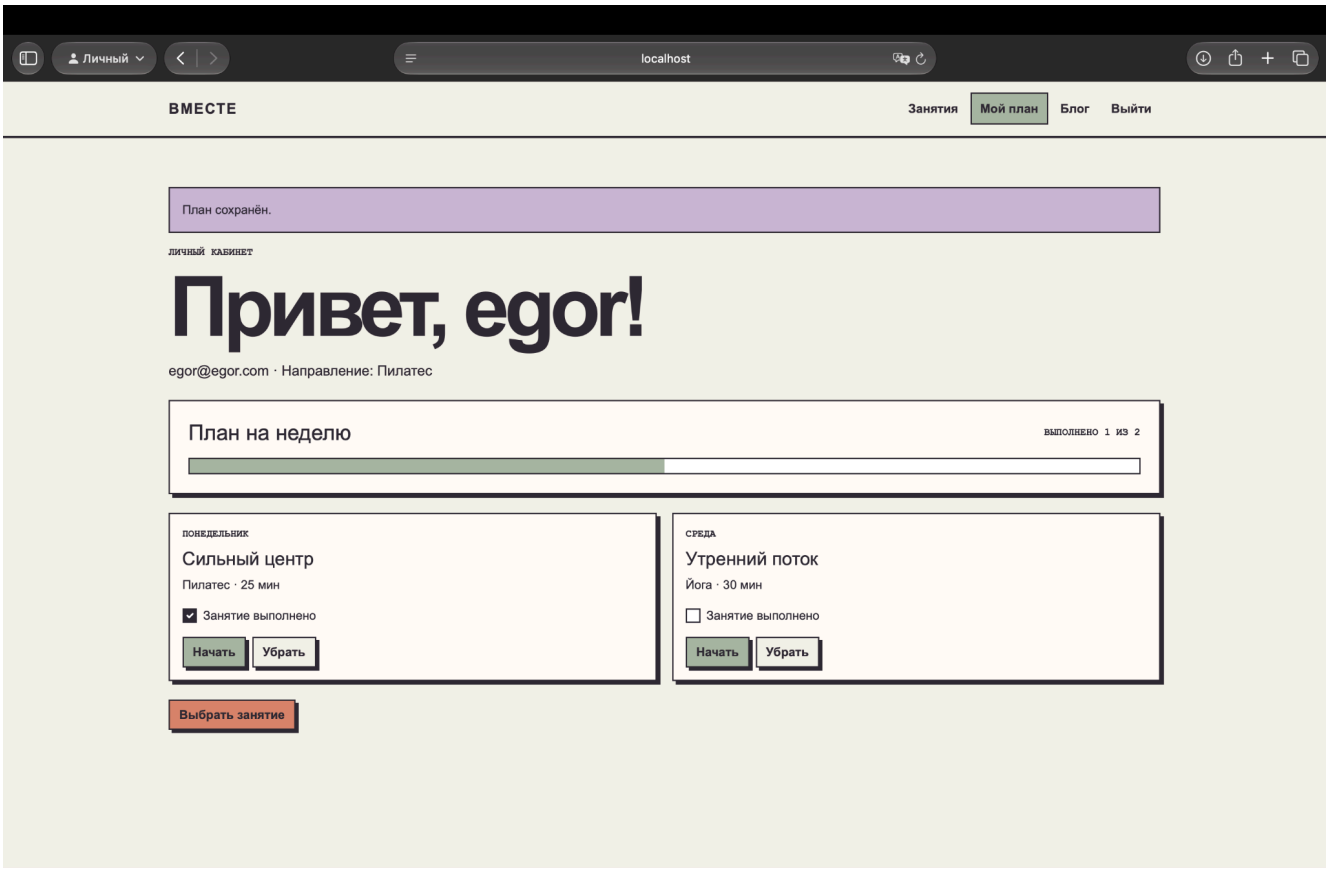


Рисунок 5. Личный план и сохранённый прогресс

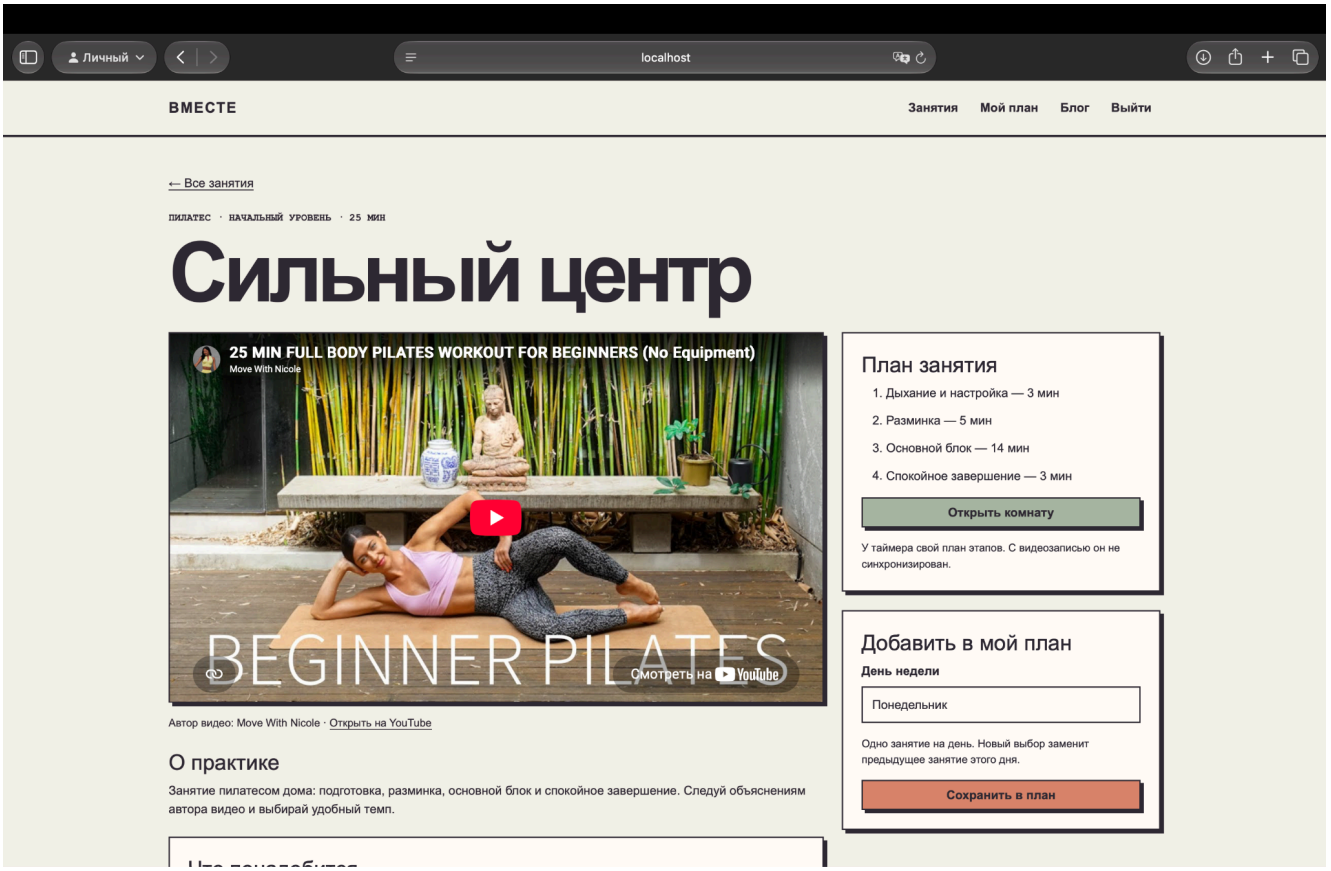


Рисунок 6. Тренировка с видео и добавлением в план

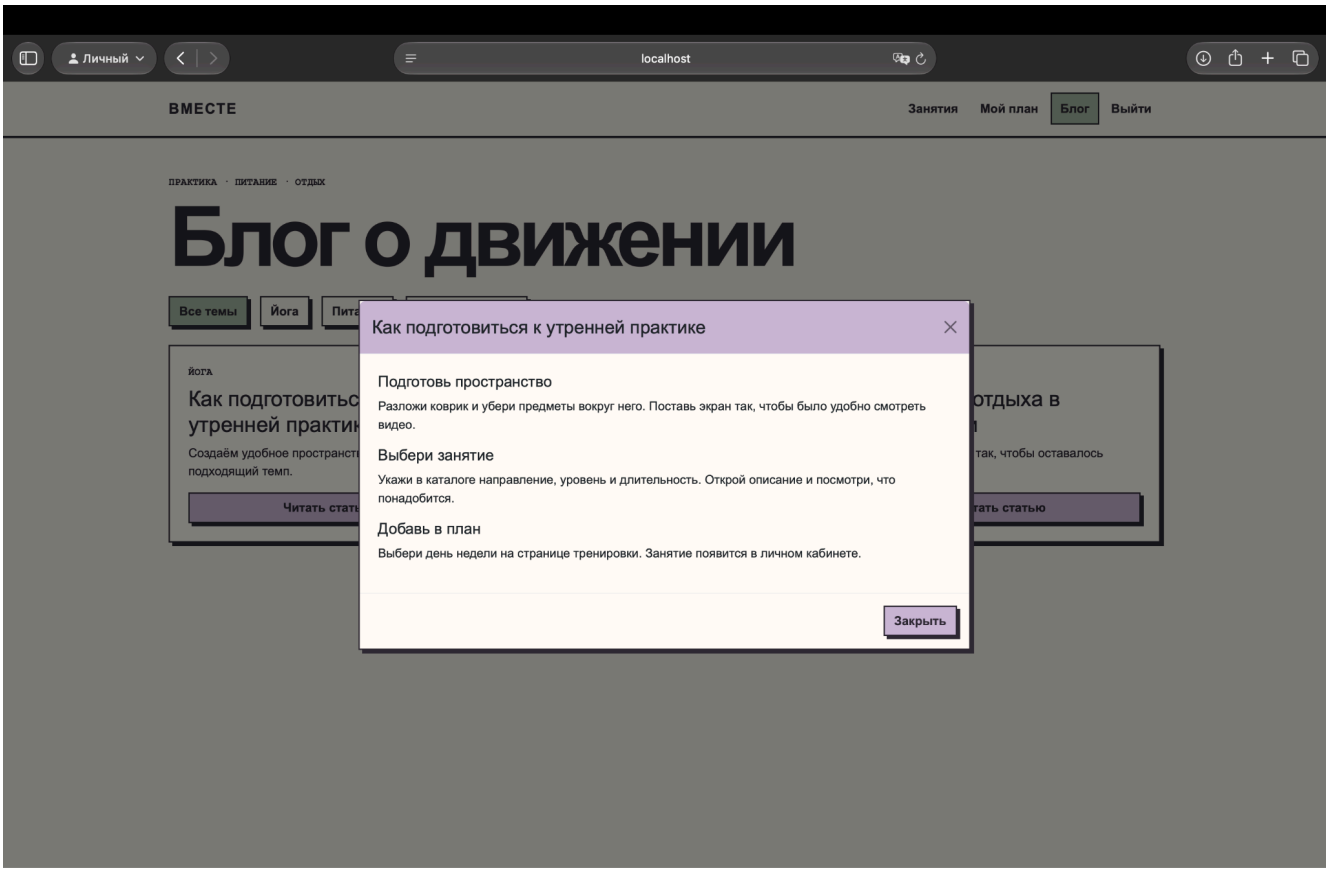


Рисунок 7. Статья из API в модальном окне

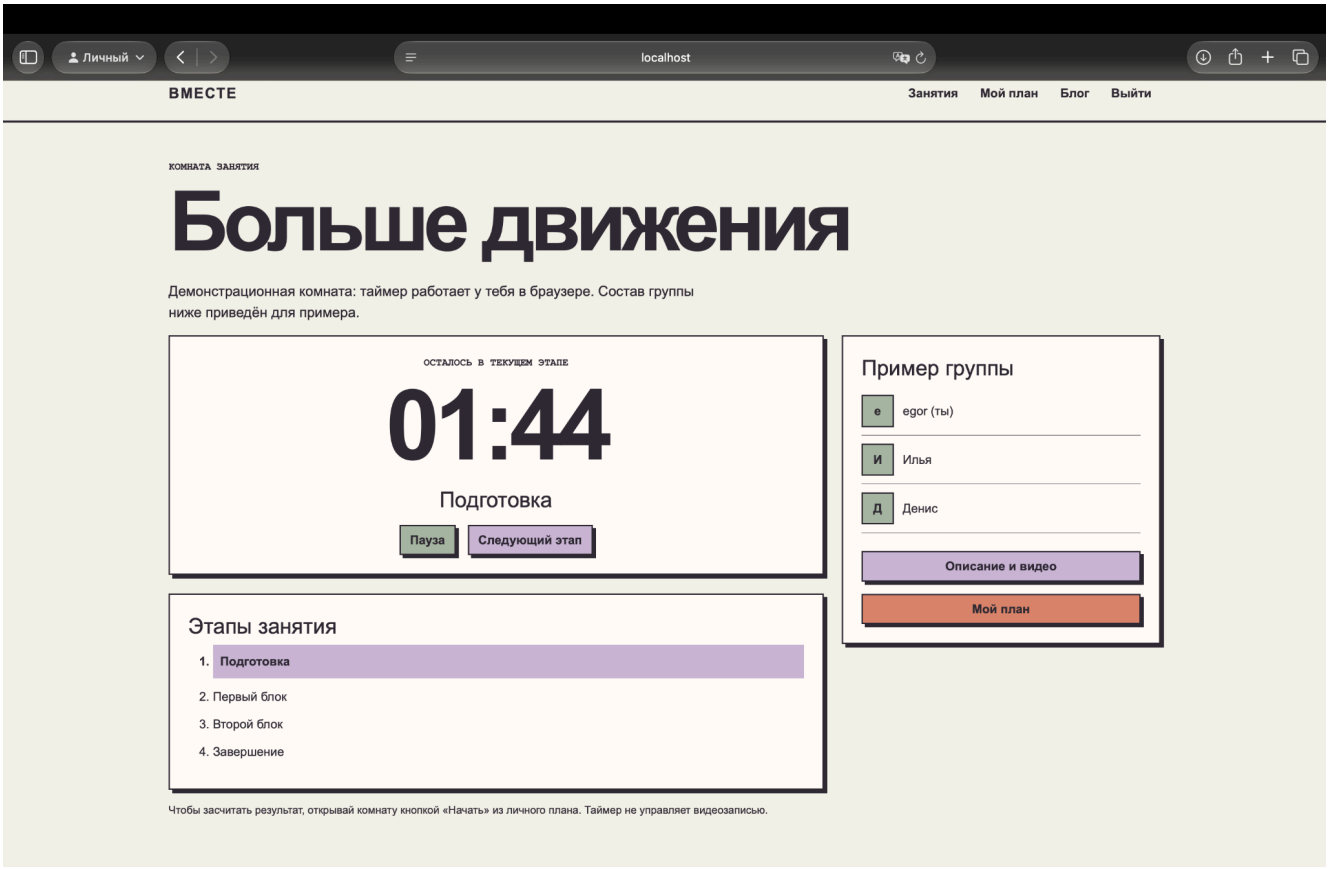


Рисунок 8. Таймер и этапы выбранного занятия

Основные фрагменты кода

Листинг 1. Подключение API и авторизации – server.cjs

```
const router = jsonServer.router(path.join(__dirname, 'db.json'));
app.db = router.db;
app.use('/api', auth.rewriter({ users: 600 }));
app.use('/api', auth);
app.use('/api', router);
```

JSON Server работает с файлом db.json. Правило users: 600 разрешает доступ к записи только её владельцу. Проверка авторизации выполняется до обработки запросов к данным.

Листинг 2. Запрос к серверу – public/js/api.js

```
response = await fetch(`/api${path}`, {
    method,
    headers,
    signal,
    cache: 'no-store',
    body: body === undefined
        ? undefined
        : JSON.stringify(body)
});
```

Метод и заголовки передаются в fetch. Тело запроса преобразуется в JSON через JSON.stringify. После получения ответа программа вызывает response.json() и использует данные на странице.

Передача токена в защищённом запросе – public/js/api.js

```
headers.Authorization =
    `Bearer ${session.accessToken}`;
```

Токен передаётся при запросах личного кабинета и сохранении плана. При ответе 401 программа очищает сохранённую авторизацию и открывает страницу входа.

Листинг 3. Параметры фильтрации – public/js/catalog.js

```
const query = new URLSearchParams();
if (form.elements.duration.value) {
  query.set('minutes_lte', form.elements.duration.value);
}
if (form.elements.search.value.trim()) {
  query.set('q', form.elements.search.value.trim());
}
const items = await api(`/trainings?${query}`);
```

В minutes_lte передаётся максимальная длительность, а в q – текст поиска. Направление и уровень передаются такими же параметрами. Сервер возвращает подходящие занятия, из которых создаются карточки.

Листинг 4. Сохранение плана – public/js/ui.js

```
export async function changePlan(update) {
    const user = await requireUser();

    return api(`/users/${user.id}`, {
        method: 'PATCH',
        auth: true,
        body: {
            plan: update(user.plan || [])
        }
    });
}
```

Сначала загружается запись вошедшего пользователя. Затем изменённый план отправляется PATCH-запросом с токеном. Поэтому добавленные занятия и отметки выполнения сохраняются после перехода между страницами.

Вывод

В результате сайт «Вместе» подключён к моковому REST API. Реализованы получение и фильтрация данных через fetch, регистрация, вход и доступ к личному плану. При работе я использовал JSON, HTTP-запросы и токен авторизации. Основные требования второй лабораторной работы выполнены.